

BRYAN CASCANTE

00

PROGRAMACIÓN ORIENTADA A OBJETOS

PROYECTO

Este único proyecto del curso se trata de construir una aplicación de prestamos donde el usuario sea capaz de crear, guardar y administrar las cosas que prefiera, esto gracias a que cada cosa que el usuario pueda agregar/administrar tiene su propio tipo y puede tener muchas categorías, estas mismas cosas pueden y van a ser prestadas a otras entidades que el usuario establezca y se guarda con fecha y hora para que también tenga la posibilidad de tener alertas sobre el mismo prestamo, por ende el programa necesita tener persistencia de datos.

Este proyecto evalúa casi todo lo visto en el curso, desde el planeamiento de cero con un diagrama de clases, pasando por la división del código con paquetes que facilitan el entendimiento del código, relaciones entre clases, hasta la creación de interfaz con java swing (una gran ayuda por cierto).

INTRODUCCIÓN

SubProblemas:

1. La planeación de las clases, métodos, atributos y relaciones que van a ser necesarias durante el desarrollo proyecto
2. La creación de métodos que resuelvan cada una de las funcionalidades que pide el proyecto (seguir el diagrama de casos de uso para la controladora).
3. Que el programa tenga persistencia de datos
4. La creación de una interfaz amigable para el usuario.
5. Poder crear un manual para que cualquier persona pueda usar el programa.

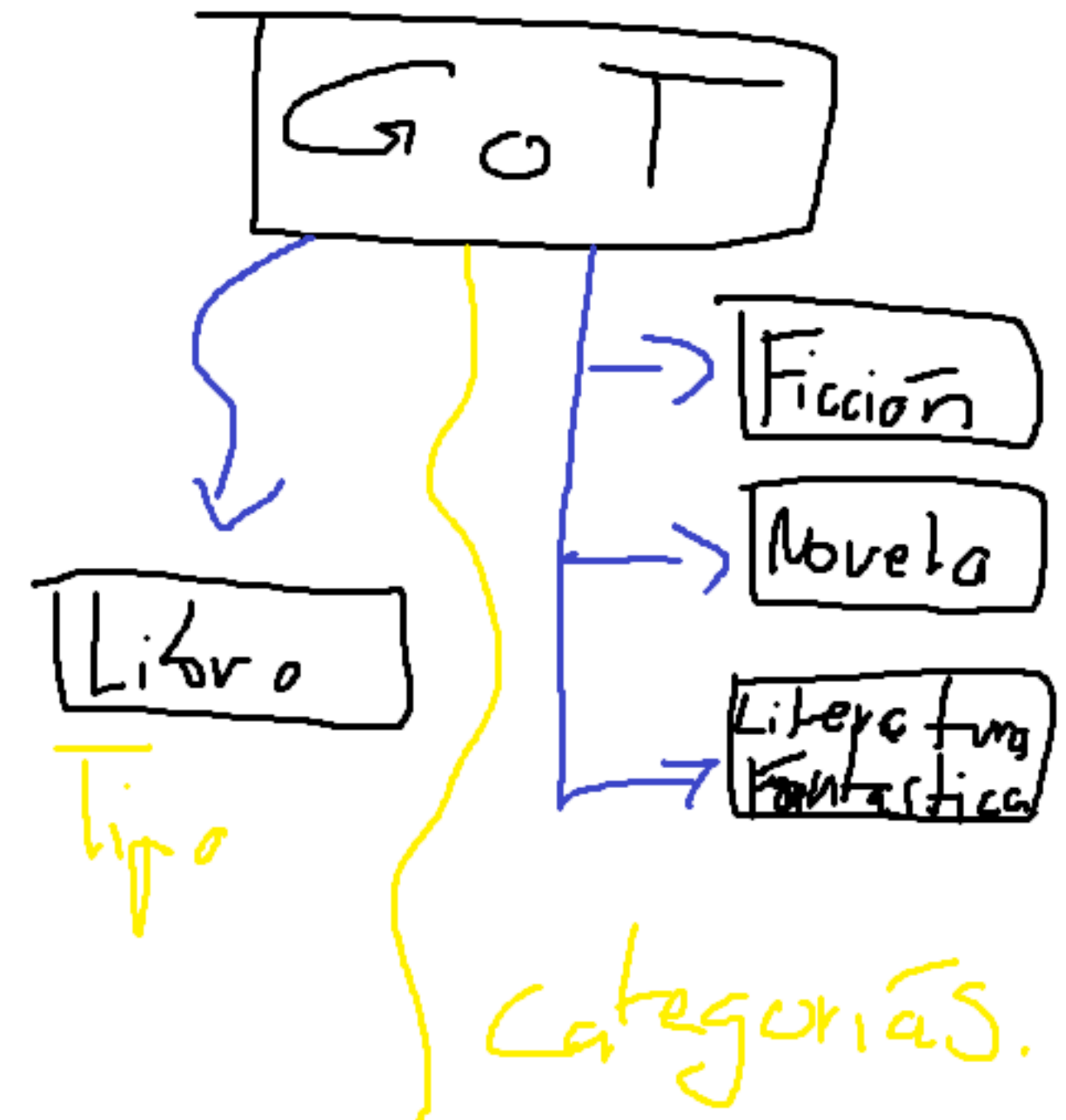
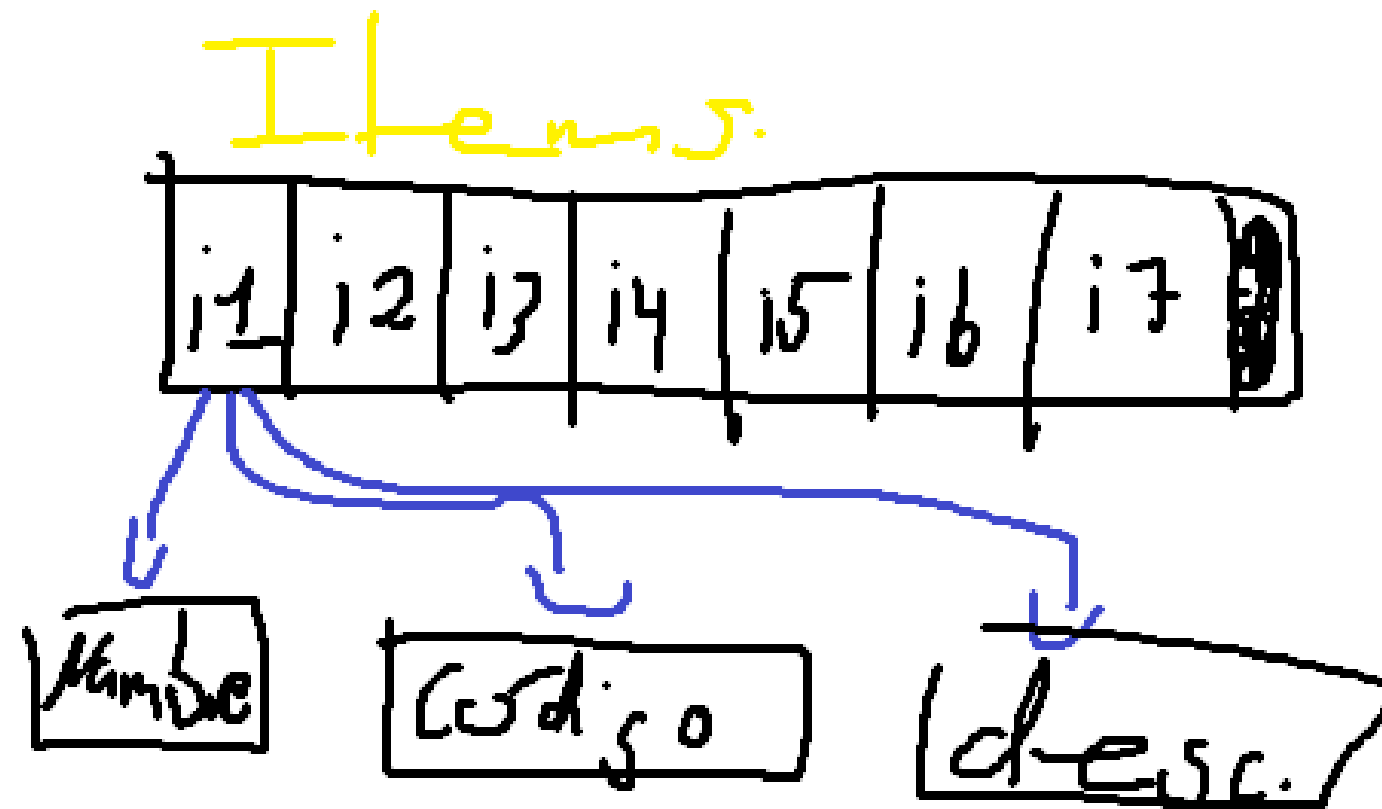


Problema principal a resolver:

El problema principal a resolver es poder hacer que varias clases, con sus respectivos arreglos de datos, puedan trabajar en conjunto, con la ayuda de otras clases que las controlan y que las pueden mostrar de manera amigable al usuario, para poder satisfacer los enunciados del proyecto.

PROBLEMAS

- 1. Para la planificación inicial, se tomaron en cuenta todas las instrucciones del proyecto, se puede decir que se iba de atrás hacia adelante, donde por medio del diagrama de usos se iba sacando que se necesitaba hacer, y después que se necesitaba para hacer lo que había que hacer, por ejemplo: analizando las instrucciones pude notar que los items iban a tener solo un tipo, pero muchas categorías.
- O que por ejemplo la parte de items, necesitaban nombre, código y descripción.



METODOLOGÍA

- 2. Para el segundo sub problema de crear los métodos de la controladora pensé que iba a ser más sencillo solo usando la lógica de las clases y que solo tenía que incluir métodos para cada botón y función, pero no fue así, conforme avancé en la creación de la interfaz me di cuenta que iba a necesitar más cosas como los métodos de listar algo, donde en el archivo principal, usé listarPrestamos() para tener un Map rápido y accesible con índices para poder hacer las cosas necesarias con un préstamo en específico. O el método de listarItemsDisponibles() que me ayudó en la parte de agregar ítems a los prestamos para poder darlos en una lista al usuario más “limpia”

```
public Map<Integer, Item> listarItemsDisponibles() {  
    Map<Integer, Item> disponibles = new HashMap<>();  
    for (Item item : items.values()) {  
        if (!itemEnPrestamo(item.getCodigo())) {  
            disponibles.put(item.getCodigo(), item);  
        }  
    }  
    return disponibles;  
}
```

```
public Map<String, Usuario> listarUsuarios() {  
    return usuarios;  
}  
  
public Map<Integer, Item> listarItems() {  
    return items;  
}  
  
public Map<String, Categoria> listarCategorias() {  
    return categorias;  
}  
  
public Map<String, Tipo> listarTipos() {  
    return tipos;  
}  
  
public Map<Integer, Prestamo> listarPrestamos() {  
    return prestamos;  
}
```

METODOLOGÍA

- 3. Para la parte de persistencia de datos lo hice como lo vimos en clase, agregué el “Serializable” a las clases de lógica, y además en la controladora agregué los métodos de cargar y guardar datos que se ejecutan cada que se abre/cierra el programa.

```
public void guardarDatos() {  
    try (ObjectOutputStream oos = new ObjectOutputStream(  
        new FileOutputStream("datos.dat"))) {  
        oos.writeObject(usuarios);  
        oos.writeObject(items);  
        oos.writeObject(categorias);  
        oos.writeObject(tipos);  
        oos.writeObject(prestamos);  
    } catch (Exception e) {  
        System.out.println("Error al guardar: " + e.getMessage());  
    }  
}
```

```
lic void cargarDatos() {  
    try (ObjectInputStream ois = new ObjectInputStream(  
        new FileInputStream("datos.dat"))) {  
        usuarios = (Map<String, Usuario>) ois.readObject();  
        items = (Map<Integer, Item>) ois.readObject();  
        categorias = (Map<String, Categoria>) ois.readObject();  
        tipos = (Map<String, Tipo>) ois.readObject();  
        prestamos = (Map<Integer, Prestamo>) ois.readObject();  
    } catch (Exception e) {  
        System.out.println("No hay datos guardados aun");  
    }  
}
```

METODOLOGÍA

- 4. La famosa interfaz, tengo que agradecer a Bijoyan Das que es un youtuber que saco hace 9 años una serie de tutoriales que me ayudaron a saber como construir la interfaz de una mejor manera, con la interfaz pasaron muchas cosas, como el hecho de usar tablas para facilitar el acceso a los datos y el uso de la misma interfaz, donde en una tabla se cargaba casi cualquier cosa y solo mediante accederla seleccionando el dato con el que queremos realizar una acción.
- Por otra parte la interfaz utiliza mucho ventanas aparte y se siguen las recomendaciones de la clase donde creamos una función privada que es la que el botón llama al momento de ser accionado, estas mismas funciones privadas son las que “realizan” realmente las tareas y en algunos casos invocan a la nueva ventana para realizar una tarea.

```
table_1 = new JTable();
table_1.setModel(new DefaultTableModel(
    new Object[][] {
    },
    new String[] {
        "ID", "Usuario", "Fecha"
    }
) {
    Class[] columnTypes = new Class[] {
        Integer.class, String.class, Object.class
    };
    public Class getColumnClass(int columnIndex) {
        return columnTypes[columnIndex];
    }
});
scrollPane_3.setViewportViewView(table_1);
```

link:https://www.youtube.com/watch?v=H63PEmEqhkM&list=PLMQ4k-hUWGNmQwP3u5HP8g4NnQQglGY_d

```
private void crearCategoria() {
    String nombre = JOptionPane.showInputDialog(frame, "ingrese el nombre de la categoria:");
    if (nombre != null && !nombre.trim().isEmpty()) {
        try {
            Controlador.getInstancia().crearCategoria(nombre.trim());
            cargarCategorias();
        } catch (Exception e) {
            JOptionPane.showMessageDialog(frame, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

METODOLOGÍA

- 5. Para el manual de usuario se enseña como usar cada función del programa.
- Este lo vamos a ver con mayor detalle al final de la presentación.

METODOLOGÍA

METODOLOGÍA

08

Colecciones usadas:

1, Map:

Categoria.items

Item.categorias

Tipo.items

Y todas las colecciones del
Controlador (usuarios, items,
categorias, tipos, prestamos)

2, ArrayList<Prestamo> es una
lista ordenada simple para que
un usuario pueda tener varios
prestamos

3, Interfaz usa un Map, como en
getItems(), getCategorias, pero se
usa como retorno de parametro

Bibliotecas usadas:

1.java.io.Serializable: que se usa
en todas las clases de logica

2.java.io.ObjectInputStream,
ObjectOutputStream,
FileInputStream,
FileOutputStream que se usan
para guardar y actualizar
datos

3.java.util.Date

4.java.util.stream

5.javax.swing

Para los patrones diría que la controladora es un singleton y tal vez
podemos ver que controlador funciona como Facade para la interfaz ya
que esta nunca accede a la logica si no que usa contolador para
modificarlos.

Algoritmos usados:

1. Busqueda lineal para recorrer
todos los prestamos

2. busqueda por clave en tablas
hash

3. Sorting de comparacion en los
reportes.

4. Serializacion de objetos en
guardarDatos() y cargarDatos()

Creo que el proyecto puede mejorar en muchas cosas, por ejemplo los comentarios que en mis codigos en general es algo que me hace mucha falta pero muchas veces no encuentro necesario ponerlos.

Por otra parte el codigo "no implementa" la interfaz como debería ya que siempre muestra todos los contenidos en tablas para cada pestaña y esa no era la idea.

A veces ocurre que cuando se vuelve a abrir el programa los datos en algunas pestañas no se muestran hasta que se vuelve a actualizar las pestaña creando o eliminando algún elemento.



Tipo	Categoria
Nombre	
importante	
tipo	
personal	
entretenimiento	
juego	

ona	Tipo	Categoria
Nombre Telefono Correo		
paula	1209380	paula@hermana
celeste	894763561	celeste@guapa
jessica	12548798	jessica@madre
Kendall	1028471024	Kendall@amigo
10		

Conclusiones:

1. Se logra crear un proyecto que logra administrar varias colecciones de objetos que persisten en memoria y cumplen el objetivo del proyecto.
2. Gracias a este proyecto se logra consolidar el conocimiento sobre el lenguaje Java y el IDE Eclipse
3. El uso del diagrama de clases me permitió tener una idea mucho más clara de como trabajar el proyecto
4. Java swing es una gran herramienta de diseño que facilitó mucho el trabajo y ayudó a que la interfaz fuera bonita
5. Gracias al debugger se lograron encontrar muchos errores y corregir fallas en el funcionamiento, además de poder ver más detalladamente como funcionaba exactamente el código.
6. Crear funciones privadas para los métodos de la interfaz permitió tener mayor claridad en el código y ayudó a ver y corregir errores
7. El desarrollo en orden desde la lógica, pasando por la controladora y por último la interfaz permitió construir código cada vez más solido y claro en todo momento.

Tipo	Categoria
Nombre	
importante	
tipo	
personal	
entretenimiento	
juego	

Persona	Tipo	Categoria
Nombre Telefono Correo		
paula	1209380	paula@hermana
celeste	894763561	celeste@guapa
jessica	12548798	jessica@madre
Kendall	1028471024	Kendall@amigo

Recomendaciones:

1. Se recomienda que para la creación de este proyecto se tome mucho en cuenta las descripciones de casos de uso, ya que en ellas está la clave para hacer muchas cosas
2. Se recomienda una amplia exploración del IDE ya que ofrece muchas herramientas que facilitan la elaboración
3. Se recomienda prestar mucho cuidado en la elaboración del diagrama, ya que es el pilar fundamental de como se va a construir el proyecto
4. Java swing es una muy buena herramienta y hay muchos tutoriales para hacer cosas en you tube, siempre es bueno repasarlos
5. Se recomienda explorar el uso del debugger paso a paso, es la mejor herramienta para saber que está fallando en el proceso
6. El uso de funciones privadas en la interfaz es la mejor manera de tener un código limpio en un archivo tan pesado
7. Se recomienda saber muy bien que tiene que hacer cada clase y que contiene, esto porque al tener claro su composición es muy sencillo crear sus métodos.



GRACIAS